

TECH

PERFORMANCE

BROWSER / RUNTIME REACT PERFORMANCE

BROWSER / RUNTIME

20 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК web.dev / developer.mozilla.org

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	CORE WEB VITALS Q1 · Q2 · Q3 · Q4	LCP	INP	CLS	Базовые метрики, по которым Google судит UX и ранжирует.
ДЕНЬ 2 СРЕДНИЙ	ИЗМЕРЕНИЯ NTECH Q5 · Q6 · Q7 · Q8	LAB vs FIELD	RUM	FI	Где брать цифры и какие важны для конкретных доменов.
ДЕНЬ 3 СРЕДНИЙ	LONG TASKS Q9 · Q10	LONG ANIMATION FRAME			Как находить блокировки main thread.
ДЕНЬ 4 ТЯЖЁЛЫЙ	RENDERING PIPELINE FORCED SYNC Q11 · Q12 · Q13	LAYOUT THRASHING			Что под капотом браузера и как не испортить себе кадры.
ДЕНЬ 5 СРЕДНИЙ	REFLOW REPAINT Q14 · Q15 · Q16	COMPOSITING			Три фазы рендера и что их триггерит.
ДЕНЬ 6 СРЕДНИЙ	ДЕШЁВЫЕ CSS-СВОЙСТВА Q17 · Q18	will-change			Что анимировать без боли и как намекнуть браузеру.
ДЕНЬ 7 СРЕДНИЙ	will-change ВПРЕД Q19 · Q20	SCROLL PERFORMANCE			Как не выстрелить себе в ногу с will-change и как ускорить scroll.
РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ					
ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ					

ДЕНЬ 1 НАГРУЗКА СРЕДНИЙ 4 ВОПРОСА

CORE WEB VITALS LCP INP CLS**ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Базовые метрики, по которым Google судит UX и ранжирует.

ВОПРОСЫ ДНЯ

Q01 Что такое Core Web Vitals?

Q02 Что такое LCP?

Q03 Что такое INP?

Q04 Что такое CLS?

Что такое Core Web Vitals?

ОПИСАНИЕ

Core Web Vitals (CWV) – три метрики Google, по которым оценивается качество UX веб-страниц: LCP (Largest Contentful Paint – скорость отрисовки), INP (Interaction to Next Paint – отзывчивость, заменил FID в марте 2024), CLS (Cumulative Layout Shift – стабильность лэйаута). Влияют на ранжирование в поиске и считаются индустриальным стандартом UX.

КАК РАБОТАЕТ

- 01 LCP: время отрисовки самого большого видимого элемента. Хорошо ≤ 2.5 с, плохо > 4 с.
- 02 INP: задержка между взаимодействием и следующей отрисовкой. Хорошо ≤ 200 мс, плохо > 500 мс.
- 03 CLS: суммарный сдвиг видимых элементов. Хорошо ≤ 0.1 , плохо > 0.25 .
- 04 Метрики собираются из реальных пользователей (CrUX) и в lab (Lighthouse).
- 05 Каждая метрика имеет 75-й перцентиль как target.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 SEO – Google использует CWV в ранжировании.
- 02 Перфоманс-аудиты перед релизом.
- 03 OKR / KPI команды frontend.
- 04 Сравнение версий между собой.

ПОДВОДНЫЕ КАМНИ

- 01 Lab-измерения не отражают реальных пользователей (см. Q6).
- 02 INP заменил FID – старые отчёты могут смотреть не туда.
- 03 Mobile-CWV обычно хуже desktop – нужно мерить отдельно.
- 04 Локальный dev-режим показывает другие цифры – тестировать production build.

КОД

// web-vitals библиотека

```
import { onLCP, onINP, onCLS } from 'web-vitals';
onLCP((m) => sendToAnalytics(m));
onINP((m) => sendToAnalytics(m));
onCLS((m) => sendToAnalytics(m));
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем INP отличается от FID?
- 02 Где смотреть real-world цифры?

03 Какой target для каждой метрики?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/vitals>

Что такое LCP?

ОПИСАНИЕ

Largest Contentful Paint – время отрисовки самого крупного видимого элемента (image, video, текстовый блок) в первом viewport. Хорошо: ≤ 2.5 секунды. Влияет на восприятие скорости загрузки. Часто узкое место – большая картинка hero-блока, медленный сервер, render-blocking JS/CSS.

КАК РАБОТАЕТ

- 01 Браузер отслеживает крупнейший элемент по площади.
- 02 `PerformanceObserver('largest-contentful-paint')`.
- 03 LCP-кандидат меняется по ходу загрузки; финальный фиксируется на первом user-interaction.
- 04 Считается от момента navigation start.
- 05 В CrUX берётся 75-й перцентиль.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая страница: новости, лендинги, продукты.
- 02 Hero-секции с картинками.
- 03 Аудиты SEO.

ПОДВОДНЫЕ КАМНИ

- 01 Картинка hero без `fetchpriority="high"` грузится поздно.
- 02 render-blocking `<script>` без `async/defer` удлиняет LCP.
- 03 Шрифты без `preload` – FOIT/FOUT.
- 04 Server-render медленный → плохой TTFB → плохой LCP.

КОД

// Приоритет hero-картинки

```

```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое TTFB и как он связан с LCP?
- 02 Как оптимизировать LCP-картинку?
- 03 Чем `preload` отличается от `prefetch`?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/lcp>

ВОПРОС Q03

Что такое INP?

ОПИСАНИЕ

Interaction to Next Paint – задержка от пользовательского взаимодействия (клик, тап, нажатие клавиши) до следующей отрисовки страницы. Заменил FID в марте 2024. Меряет всё взаимодействие за всю сессию, не только первое. Хорошо: $\leq 200\text{мс}$. Большие тяжёлые обработчики, синхронные парсинги – главные враги.

КАК РАБОТАЕТ

- 01 Время = event handler + processing + next paint.
- 02 Считается 98-й перцентиль взаимодействий.
- 03 PerformanceObserver('event') + buffered.
- 04 Худшее взаимодействие за сессию идёт в CrUX.
- 05 Long task > 50мс блокирует обработку – вредит INP.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Сложные дашборды.
- 02 Формы с тяжёлой валидацией.
- 03 Аналитика в обработчиках.
- 04 SPA с большим JS-bundle.

ПОДВОДНЫЕ КАМНИ

- 01 Синхронные обработчики >50мс – INP плохой.
- 02 Аналитика в onClick без отложения – задержка.
- 03 Большой setState в обработчике без useTransition.
- 04 Re-render всего дерева при простом действии.

КОД

// Отложенная аналитика

```
function onClick() {
  doImportant();
  // не блокировать interaction
  scheduler.postTask(() => track('click'), { priority: 'background' });
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем INP отличается от FID?
- 02 Что такое long task?
- 03 Как измерить INP в проде?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/inp>

ВОПРОС Q04

Что такое CLS?

ОПИСАНИЕ

Cumulative Layout Shift – сумма всех неожиданных сдвигов видимых элементов на странице. Хорошо: ≤ 0.1 . Главные виновники: картинки без размеров, ленивые шрифты, рекламные блоки, баннеры cookie consent, поздняя hydration-разметка. Каждый сдвиг считается как $\text{impact_fraction} \times \text{distance_fraction}$.

КАК РАБОТАЕТ

- 01 Сдвиг = unexpected position-change of visible element.
- 02 Не считаются: user-initiated (клик/анимация в ответ).
- 03 Окно сессии: 1 секунда между сдвигами, max 5 секунд.
- 04 Сумма по сессии – CLS.
- 05 PerformanceObserver('layout-shift').

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая страница с медиа-контентом.
- 02 Лендинги с баннерами.
- 03 SSR-приложения с hydration.

ПОДВОДНЫЕ КАМНИ

- 01 `<img>` без width/height – сдвиг при загрузке.
- 02 Шрифты без font-display: optional → FOUT/FOIT и сдвиг.
- 03 Реклама/баннеры без зарезервированной высоты.
- 04 Skeleton без точных размеров.

КОД

// Резервирование места

```

<!-- браузер сразу резервирует aspect-ratio -->
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое font-display?
- 02 Как зарезервировать место под динамику?
- 03 Какие сдвиги не считаются в CLS?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/cls>

ИЗМЕРЕНИЯ**LAB vs FIELD****RUM****FINTECH****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Где брать цифры и какие важны для конкретных доменов.

ВОПРОСЫ ДНЯ

Q05 Как измерять frontend performance?

Q06 Чем lab metrics отличаются от field metrics?

Q07 Что такое RUM?

Q08 Какие performance-метрики важны для trading/fintech UI?

ВОПРОС Q05

Как измерять frontend performance?

ОПИСАНИЕ

Несколько источников: Lighthouse / PageSpeed Insights (lab-измерения), Chrome DevTools Performance / Performance Insights (lab + tracing), web-vitals библиотека (RUM в проде), CrUX BigQuery (Google data из Chrome пользователей), коммерческие RUM (SpeedCurve, Calibre, Datadog Real User Monitoring). В идеале – всё вместе.

КАК РАБОТАЕТ

- 01 Lab – синтетические тесты в контролируемой среде.
- 02 Field – реальные пользователи (RUM).
- 03 Lighthouse: запускается локально или в CI.
- 04 Performance API в браузере – встроенные таймеры.
- 05 web-vitals – библиотека для отправки CWV в analytics.
- 06 Synthetic monitoring – крон-проверки из дата-центров.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 В CI – Lighthouse-чек на каждый PR.
- 02 В RUM – постоянный мониторинг production.
- 03 При баг-репортах – Performance trace конкретного сценария.
- 04 На уровне OKR – отслеживать метрики quarter-over-quarter.

ПОДВОДНЫЕ КАМНИ

- 01 Лаба не отражает реальных пользователей (slow 3G, old phones).
- 02 RUM без сегментации показывает «среднюю температуру».
- 03 Лайтхаус с разной hardware показывает разные цифры.
- 04 Без budget – метрики никто не контролирует.

КОД

// RUM через web-vitals

```
import { onLCP, onINP, onCLS } from 'web-vitals';
[onLCP, onINP, onCLS].forEach((fn) =>
  fn((m) => navigator.sendBeacon('/log', JSON.stringify(m))));
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем lab от field отличается?
- 02 Что такое CrUX?
- 03 Какие RUM-сервисы популярны?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/user-centric-performance-metrics>

ВОПРОС Q06

Чем lab metrics отличаются от field metrics?

ОПИСАНИЕ

Lab – измерения в контролируемой среде: одна машина, одна сеть, повторяемые условия. Lighthouse – канонический пример. Подходит для CI, debug, regression-тестов. Field (RUM, real-user-monitoring) – измерения у реальных пользователей: разные устройства, сети, локации. Подходит для понимания «что видят пользователи». Часто значения сильно отличаются.

КАК РАБОТАЕТ

- 01 Lab: фиксированный environment, повторяемо.
- 02 Field: реальные данные, шумные.
- 03 Lab измеряет потенциал; field – фактический UX.
- 04 CrUX (Chrome User Experience Report) – публичный field-источник.
- 05 Лучшее: сочетать оба – CI с lab, прод с RUM.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Lab – debug, CI, regression.
- 02 Field – мониторинг прода, customer-facing метрики.
- 03 Для optimisations: lab – гипотеза, field – проверка.

ПОДВОДНЫЕ КАМНИ

- 01 Только lab – не видно, что у пользователей.
- 02 Только field – нельзя воспроизвести проблему.
- 03 Lab на быстрой машине маскирует проблемы.
- 04 Field без сегментации не показывает корень.

КОД

// CrUX через PageSpeed API

```
fetch(`https://www.googleapis.com/pagespeedonline/v5/runPagespeed?url=${url}&strategy=m
obile`)
  .then((r) => r.json())
  .then((d) => log(d.loadingExperience.metrics));
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое CrUX?
- 02 Чем Lighthouse отличается от PageSpeed?
- 03 Какие RUM-провайдеры?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/lab-and-field-data-differences>

ВОПРОС Q07

Что такое RUM?

ОПИСАНИЕ

Real User Monitoring – сбор перфоманс-метрик у реальных пользователей в продакшене. Включает CWV, long tasks, errors, custom timings. Реализуется встраиванием JS-snippet (web-vitals + sendBeacon своими силами) или коммерческими сервисами (SpeedCurve, Datadog RUM, NewRelic Browser, Sentry Performance).

КАК РАБОТАЕТ

- 01 JS-агент встроен в страницу, собирает метрики.
- 02 Отправляет через sendBeacon (не блокирует).
- 03 Сегментация: по странице, по платформе, по гео.
- 04 Дашборды: 75-й, 95-й перцентили, alerts при регрессиях.
- 05 Корреляция с release-метками – для post-mortem.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Production-мониторинг.
- 02 Customer-facing SLA.
- 03 A/B-эксперименты – сравнение групп.
- 04 Корреляция перфоманса с бизнес-метриками.

ПОДВОДНЫЕ КАМНИ

- 01 Без сегментации – "средняя температура".
- 02 RUM сам себе adds overhead – мониторить.
- 03 Sample rate – на больших масштабах.
- 04 GDPR – собираем data о пользователях.

КОД

// Минимальный RUM

```
import { onLCP, onINP, onCLS } from 'web-vitals';
function send(metric) {
  const body = JSON.stringify({ ...metric, url: location.pathname });
  navigator.sendBeacon('/api/rum', body);
}
onLCP(send); onINP(send); onCLS(send);
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие segment-keys полезны?
- 02 Чем sendBeacon лучше fetch?
- 03 Как защитить privacy?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/rum>

ВОПРОС Q08

Какие performance-метрики важны для trading/fintech UI?

ОПИСАНИЕ

Помимо CwV – реактивность под высоким потоком данных: INP при частых обновлениях, длительность frame (FPS), latency от tick до отображения, время обработки одного market-update, время от клика «купить» до подтверждения. Tearing и stale data критичны: пользователь видит «свежее» в одной колонке и старое в другой.

КАК РАБОТАЕТ

- 01 INP / Long Animation Frames – отзывчивость.
- 02 Tick-to-render latency: timestamp source vs paint.
- 03 FPS под нагрузкой (60 на десктопе, 30 на mobile).
- 04 Throughput WebSocket-сообщений.
- 05 P99 для submit-button → ack.
- 06 Memory leak rate – длинные сессии.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Биржевые/фин-стримы.
- 02 Realtime dashboards.
- 03 Trading desktop-like web-приложения.

ПОДВОДНЫЕ КАМНИ

- 01 Стандартный CwV не показывает rendering под нагрузкой.
- 02 Без virtualization больших стаканов – kill UI.
- 03 Лишний context-update на каждый tick – re-render storm.
- 04 Typing pop-up котировок без useDeferredValue.

КОД

```
// Tick-to-render
```

```
function onTick(t) {
  const sentAt = t.ts;
  requestAnimationFrame(() => {
    log('tick-to-paint', performance.now() - sentAt);
    update(t);
  });
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое tearing в realtime UI?
- 02 Какие FPS критичны для trading?

03 Как профилировать high-frequency UI?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/inp>

ДЕНЬ 3 НАГРУЗКА СРЕДНИЙ 2 ВОПРОСА

LONG TASKS

LONG ANIMATION FRAME

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Как находить блокировки main thread.

ВОПРОСЫ ДНЯ

Q09 Как найти long task?

Q10 Как найти long animation frame?

Как найти long task?

ОПИСАНИЕ

Long task – синхронная задача в main thread > 50мс. Блокирует input, рендер, анимации. Найти: Chrome DevTools Performance (запись + анализ), Performance Insights, PerformanceObserver('longtask'). RUM-библиотеки тоже регистрируют. Цель – разбить тяжёлый код на куски через scheduler.postTask, Web Worker, или break-up через requestIdleCallback.

КАК РАБОТАЕТ

- 01 PerformanceObserver({ entryTypes: ['longtask'] }).
- 02 Entry содержит startTime, duration, attribution.
- 03 В DevTools: Performance → Main → жёлтые «Tasks».
- 04 scheduler.postTask({ priority }) – для разбивки.
- 05 Web Worker – для CPU-тяжёлых операций.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 INP regression – обычно из-за long task.
- 02 Анимации лагают.
- 03 Click → response delay > 200мс.
- 04 Profiler во время load.

ПОДВОДНЫЕ КАМНИ

- 01 Long task в setTimeout – браузер всё равно блокируется.
- 02 JSON.parse больших ответов – типичный long task.
- 03 Аналитика-batches без yield → один большой блок.
- 04 Чрезмерная синхронность в hydration.

КОД

// Регистрация long task

```
new PerformanceObserver((list) => {
  for (const e of list.getEntries()) {
    if (e.duration > 50) console.warn('long task', e);
  }
}).observe({ entryTypes: ['longtask'] });
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое scheduler.postTask?
- 02 Как разбить тяжёлый for-loop?
- 03 Чем yield to main thread помогает?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/long-tasks-devtools>

Как найти long animation frame?

ОПИСАНИЕ

Long Animation Frame (LoAF) API – расширение Long Tasks: сообщает о слишком длинных кадрах рендеринга, включая render-фазу, не только JS. Полезно для понимания, что именно блокирует – JS, layout, paint, или style recalculation. Доступно в Chrome 123+.

КАК РАБОТАЕТ

- 01 PerformanceObserver({ entryTypes: ['long-animation-frame'] }).
- 02 Entry: startTime, duration, scripts (массив скриптов с их временем), renderStart, blockingDuration.
- 03 Дополняет longtask – показывает не только JS, но и render.
- 04 Помогает выявить тяжёлые layout/paint, не только JS.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Анимации лагают, а long-task пуст.
- 02 Сложные сцены canvas / SVG.
- 03 Тяжёлые reflow при scroll.
- 04 Хочется атрибутировать – какой скрипт виноват.

ПОДВОДНЫЕ КАМНИ

- 01 Поддержка пока только Chromium – не везде.
- 02 Большой объём данных на тяжёлой странице.
- 03 Не на каждый кадр – только «long».
- 04 Нужен fallback на longtask для других браузеров.

КОД

// LoAF

```
new PerformanceObserver((list) => {
  list.getEntries().forEach((e) => {
    console.log('LoAF', e.duration, e.scripts);
  });
}).observe({ entryTypes: ['long-animation-frame'] });
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем LoAF отличается от longtask?
- 02 Что такое blockingDuration?
- 03 Где смотреть scripts attribution?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/long-animation-frames>

RENDERING PIPELINE**LAYOUT THRASHING****FORCED SYNC****ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Что под капотом браузера и как не испортить себе кадры.

ВОПРОСЫ ДНЯ

Q11 Как работает browser rendering pipeline?

Q12 Что такое layout thrashing?

Q13 Как избежать forced synchronous layout?

ВОПРОС Q11

Как работает browser rendering pipeline?

ОПИСАНИЕ

Pipeline: JavaScript → Style (recalc) → Layout (reflow) → Paint → Composite. Изменение CSS-свойства триггерит разный объём работы. Transform/opacity – только composite (дешёвые). width/top/left – layout + paint + composite (дорогие). Понимание pipeline помогает выбирать дешёвые анимации и избегать reflow-storm.

КАК РАБОТАЕТ

- 01 1) JS – обновляет DOM/CSS.
- 02 2) Style – пересчёт computed styles.
- 03 3) Layout – расчёт positions/sizes.
- 04 4) Paint – отрисовка пикселей.
- 05 5) Composite – сборка слоёв на GPU.
- 06 Каждое изменение начинается с какого-то шага и идёт вниз.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Анимации – выбор дешёвых свойств.
- 02 Оптимизация скролла.
- 03 Дебаг "страница лагает".
- 04 Разработка canvas / WebGL.

ПОДВОДНЫЕ КАМНИ

- 01 Анимация width/top → reflow на каждый кадр.
- 02 Forced sync layout (см. Q13) – сразу два прохода.
- 03 Большой DOM делает layout дороже.
- 04 Composite слишком много слоёв – память GPU.

КОД

// Дёшево vs дорого

```
/* дешёво – composite only */
transform: translateX(100px);
opacity: 0.5;

/* дорого – layout + paint */
left: 100px;
width: 200px;
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое compositor thread?

02 Какие свойства триггерят только paint?

03 Что такое will-change?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/Performance/How_browsers_work

ВОПРОС Q12

Что такое layout thrashing?

ОПИСАНИЕ

Layout thrashing – повторяющееся чередование read/write DOM в одном тике, из-за чего браузер вынужден делать layout СИНХРОННО на каждом read. Классика: цикл, где читаем offsetHeight и тут же меняем style. Каждое чтение требует актуального layout, а предыдущая запись его invalidate. Лекарство: сначала все чтения, потом все записи, или использовать requestAnimationFrame.

КАК РАБОТАЕТ

- 01 Браузер batch'ит изменения и пересчитывает layout раз в конце тика.
- 02 Если запросить геометрию (offsetHeight, getBoundingClientRect) после write – layout пересчитывается прямо сейчас.
- 03 Цикл «write → read → write → read» = N reflow.
- 04 FastDOM / lib-batched чтения.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Циклы по элементам с измерением размеров.
- 02 Drag-and-drop с измерением target.
- 03 Виртуализация без оптимизации.
- 04 Animation libraries без батчинга.

ПОДВОДНЫЕ КАМНИ

- 01 Forced sync layout – invisible, но drains performance.
- 02 В DevTools отмечается фиолетовым «Recalculate Style» + «Layout».
- 03 В скролле – катастрофа.

КОД

// Антипаттерн

```
// плохо: thrashing
for (const el of items) {
  const w = el.offsetWidth; // read → forces layout
  el.style.width = (w + 10) + 'px'; // write → invalidate
}

// хорошо: read all, then write all
const widths = items.map((el) => el.offsetWidth);
items.forEach((el, i) => {
  el.style.width = (widths[i] + 10) + 'px';
});
```

МОГУТ ДОСПРОСИТЬ

- 01 Что вызывает forced sync layout?
- 02 Чем requestAnimationFrame помогает?
- 03 Что такое FastDOM?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/avoid-large-complex-layouts-and-layout-thrashing>

ВОПРОС Q13

Как избежать forced synchronous layout?

ОПИСАНИЕ

Forced sync layout – когда JS заставляет браузер синхронно пересчитать layout до планового момента. Триггеры – чтение свойств, требующих актуальный layout (offsetHeight, scrollTop, getBoundingClientRect, getComputedStyle). Лекарство: батчить чтения и записи, использовать ResizeObserver/IntersectionObserver вместо синхронных read'ов.

КАК РАБОТАЕТ

- 01 Browser отложил layout до конца тика.
- 02 Read свойства → принудительно делает layout сейчас.
- 03 Write CSS → invalidate.
- 04 Read after write → ещё один forced layout.
- 05 ResizeObserver – асинхронно, не вызывает sync.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Везде, где есть ручные измерения DOM.
- 02 Анимации с измерениями.
- 03 Auto-resize textarea / table.
- 04 Sticky headers.

ПОДВОДНЫЕ КАМНИ

- 01 Получение computed style – тоже forces layout.
- 02 Чтение в цикле – много forced layouts.
- 03 Зависимости от точных размеров в обработчике scroll.
- 04 В React StrictMode dev двойной render → двойные измерения.

КОД

// Read → write порядок

```
const widths = items.map(el => el.offsetWidth);
items.forEach((el, i) => el.style.width = `${widths[i]}px`);
// один layout на всё
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие свойства forces layout?
- 02 Как ResizeObserver помогает?
- 03 Что такое DOM-batched libraries?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/avoid-large-complex-layouts-and-layout-thrashing>

ДЕНЬ 5 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

REFLOW REPAINT COMPOSITING**ЧТО УЧИМ В ЭТОТ ДЕНЬ**

Три фазы рендера и что их триггерит.

ВОПРОСЫ ДНЯ

Q14 Что вызывает reflow?

Q15 Что вызывает repaint?

Q16 Что такое compositing?

Что вызывает reflow?

ОПИСАНИЕ

Reflow (layout) – перерасчёт positions/sizes элементов. Вызывают: изменения геометрии (width, height, padding, margin, top, font-size), добавление/удаление DOM-узлов, изменение содержимого (innerText), запросы геометрии (offsetHeight, getBoundingClientRect). Resize окна – глобальный reflow.

КАК РАБОТАЕТ

- 01 Geometric CSS changes: width/height/padding/margin/top/left/font-size.
- 02 DOM mutations: appendChild, removeChild, insertBefore.
- 03 Content changes: innerText, innerHTML.
- 04 Geometric reads: offsetX, scrollX, getBoundingClientRect.
- 05 Window resize → global reflow.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Понимание помогает писать дешёвые анимации.
- 02 Дебаг "страница лагает".
- 03 Оптимизация SSR-hydration.

ПОДВОДНЫЕ КАМНИ

- 01 Анимация width – reflow каждый кадр.
- 02 Изменение font-size в loop – reflow на каждое.
- 03 innerHTML большого блока – reflow всего.
- 04 Forced sync layout – двойной reflow.

КОД

// Что reflow, что не

```
/* reflow */
el.style.width = '100px';
el.style.padding = '10px';
el.appendChild(child);

/* HE reflow (только paint/composite) */
el.style.color = 'red';
el.style.transform = 'translateX(10px)';
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем layout от paint отличается?
- 02 Можно ли анимировать размеры дешево?

03 Что такое containment?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Glossary/Reflow>

Что вызывает repaint?

ОПИСАНИЕ

Repaint – перерисовка пикселей без изменения геометрии. Триггеры: цвет, background, box-shadow, outline, visibility. Repaint дешевле reflow, но всё равно стоит CPU/GPU. Composite-only свойства (transform, opacity) дешевле repaint, потому что не перерисовывают пиксели.

КАК РАБОТАЕТ

- 01 color, background, border-color, box-shadow, outline → paint.
- 02 visibility: hidden/visible → paint, не reflow.
- 03 Текст-выделение → paint.
- 04 Currently-painted layer перерисовывается.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Hover-эффекты на цвет.
- 02 Smooth color transitions.
- 03 Theme switch.

ПОДВОДНЫЕ КАМНИ

- 01 Тяжёлый box-shadow на тысяче элементов – paint медленный.
- 02 Background-image на скролле без containment.
- 03 Большой painted layer – память.

КОД

// Paint vs composite

```
/* paint */
.btn:hover { background: blue; }

/* composite-only */
.btn:hover { transform: scale(1.05); }
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем paint от composite отличается?
- 02 Какие свойства composite-only?
- 03 Что такое contain CSS-property?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/Performance/How_browsers_work

ВОПРОС Q16

Что такое compositing?

ОПИСАНИЕ

Composite — финальная фаза рендера: GPU собирает несколько layers в финальное изображение. Layer — это часть страницы, отрисованная отдельно (как слой в Photoshop). Свойства transform/opacity меняются только в этой фазе — без повторного paint. Это дешево и плавно. Поэтому анимации лучше делать через transform.

КАК РАБОТАЕТ

- 01 Page разбивается на layers — каждый рисуется отдельно.
- 02 GPU собирает финальный bitmap.
- 03 transform/opacity меняются только на GPU — без paint.
- 04 Layer создаётся для: position fixed, transform non-zero, will-change, video, canvas.
- 05 Слишком много layer'ов = память GPU.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Анимации position через transform.
- 02 Modal-overlay через opacity.
- 03 Драгабельные элементы.
- 04 Smooth scroll-эффекты.

ПОДВОДНЫЕ КАМНИ

- 01 will-change без причины — лишние layers, GPU память.
- 02 Слишком много фиксированных элементов = много layers.
- 03 Composite не «дешёвый» — у GPU тоже свои лимиты.

КОД

// Composite-only анимация

```
@keyframes slide {
  from { transform: translateX(0); opacity: 1; }
  to   { transform: translateX(100px); opacity: 0; }
}
.item { animation: slide 200ms ease; }
```

МОГУТ ДОСПРОСИТЬ

- 01 Что создаёт новый composite layer?
- 02 Как посмотреть layers в DevTools?
- 03 Чем will-change от transform отличается?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Glossary/Compositing>

ДЕНЬ 6 НАГРУЗКА СРЕДНИЙ 2 ВОПРОСА

ДЕШЁВЫЕ CSS-СВОЙСТВА `will-change`

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Что анимировать без боли и как намекнуть браузеру.

ВОПРОСЫ ДНЯ

Q17 Какие CSS-свойства дешевле анимировать?

Q18 Когда использовать `will-change`?

Какие CSS-свойства дешевле анимировать?

ОПИСАНИЕ

Дешёвые: transform (translate, scale, rotate), opacity. Они работают только в composite-фазе, не вызывают reflow и paint. Дорогие: width, height, top, left, padding, margin, font-size – каждый кадр reflow + paint. Если анимация лагает – заменить геометрические свойства на эквивалент через transform.

КАК РАБОТАЕТ

- 01 transform: translate/scale/rotate – composite only.
- 02 opacity – composite only.
- 03 filter – paint, но без reflow.
- 04 background-color – paint.
- 05 width/height/top/left → reflow + paint.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 UI-анимации (slide, fade, scale).
- 02 Drag-drop position changes.
- 03 Hover-эффекты.
- 04 Page transitions.

ПОДВОДНЫЕ КАМНИ

- 01 scale меняет визуальный размер – но не layout. Соседние элементы не двигаются.
- 02 translateY ≠ top: layer-aware.
- 03 Многослойные анимации могут переполнить GPU memory.

КОД

// Замена top на translate

```
/* плохо */
@keyframes slide { from { top: 0; } to { top: 100px; } }

/* хорошо */
@keyframes slide { from { transform: translateY(0); } to { transform: translateY(100px); } }
```

МОГУТ ДОСПРОСИТЬ

- 01 Чем translateY от top отличается?
 - 02 Какой эффект у filter: blur?
 - 03 Что такое CSS containment?
-

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/animations-guide>

Когда использовать will-change?

ОПИСАНИЕ

will-change подсказывает браузеру, какие свойства будут анимироваться, чтобы заранее создать composite layer. Используется ТОЧЕЧНО – для элементов, которые точно будут анимированы. Снимать после анимации (will-change: auto). Без меры – наоборот вредит (см. Q19).

КАК РАБОТАЕТ

- 01 will-change: transform, opacity – браузер готовит layer.
- 02 Effective использование – на элементах, которые анимируются по hover/click.
- 03 Снимается через will-change: auto.
- 04 Альтернативы – translateZ(0) хак (forces layer), но will-change цивилизованнее.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Drag-drop – на drag-target.
- 02 Hover-эффекты с transform.
- 03 Открывающиеся drawer / modal.
- 04 Скролл-параллакс.

ПОДВОДНЫЕ КАМНИ

- 01 На каждом элементе – трата GPU-памяти.
- 02 Без снятия – layer навсегда.
- 03 Не для всех свойств работает – только для composite-way.
- 04 В Safari иногда бесполезен.

КОД

// Точечно

```
.draggable { will-change: transform; }  
/* после finishing – снять */  
el.style.willChange = 'auto';
```

МОГУТ ДОСПРОСИТЬ

- 01 Какие свойства имеет смысл указать?
- 02 Что такое translateZ(0) хак?
- 03 Когда will-change бесполезен?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/CSS/will-change>

ДЕНЬ 7

НАГРУЗКА

СРЕДНИЙ

2 ВОПРОСА

will-change ВРЕД SCROLL PERFORMANCE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

Как не выстрелить себе в ногу с will-change и как ускорить scroll.

ВОПРОСЫ ДНЯ

Q19 Почему will-change может навредить?

Q20 Как оптимизировать scroll performance?

Почему will-change может навредить?

ОПИСАНИЕ

Если поставить will-change на много элементов или на всё подряд, браузер создаст много composite-layers, что отъест GPU-память и может привести к просадке производительности. Кроме того, layer навсегда «выделен» – даже когда анимация не идёт. Нужно использовать will-change ТОЧЕЧНО и снимать после.

КАК РАБОТАЕТ

- 01 Каждый will-change-элемент создаёт layer.
- 02 Layer = bitmap в GPU memory.
- 03 Много layers – нагрузка на GPU и больше работы для compositor.
- 04 На mobile с ограниченной памятью – особенно вредно.
- 05 Без снятия – layer не освобождается.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Понимание помогает не злоупотреблять.
- 02 Code review – флагать will-change на всём.
- 03 Profiling GPU memory.

ПОДВОДНЫЕ КАМНИ

- 01 .item { will-change: transform; } для list из 10000 = катастрофа.
- 02 will-change в hover-state – браузер уже не успевает подготовить.
- 03 На static-элементах – мёртвая память.

КОД

// Антипаттерн

```
.item { will-change: transform; } /* для всех 1000 – плохо */

/* лучше – только в момент drag */
.item.dragging { will-change: transform; }
```

МОГУТ ДОСПРОСИТЬ

- 01 Когда снимать will-change?
- 02 Что такое layer count?
- 03 Как проверить GPU usage?

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/CSS/will-change>

ВОПРОС Q20

Как оптимизировать scroll performance?

ОПИСАНИЕ

Принципы: passive event listeners (см. Q8 в browser-apis-dom), не делать reflow в обработчике scroll, использовать requestAnimationFrame для обновлений UI, content-visibility: auto / contain для off-screen контента, виртуализация длинных списков, GPU-accelerated анимации (transform), избегать backdrop-filter / box-shadow на крупных элементах.

КАК РАБОТАЕТ

- 01 passive: true для scroll listeners.
- 02 Throttle/raf обновления внутри scroll handler.
- 03 content-visibility: auto – браузер skip-ает off-screen.
- 04 contain: layout/paint – изоляция.
- 05 Виртуализация (react-window) для больших списков.
- 06 Composite-only animations.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Длинные ленты (соцсети, каталоги).
- 02 Дашборды с тяжёлыми виджетами.
- 03 Tables с большим количеством строк.
- 04 Mobile-веб с touch-scroll.

ПОДВОДНЫЕ КАМНИ

- 01 JS-handler в scroll, который меняет style – janky scroll.
- 02 Тяжёлый box-shadow / backdrop-filter – рисуется каждый кадр.
- 03 Без content-visibility – браузер рисует всё, что в DOM.
- 04 Sticky headers с reflow внутри.

КОД

// Scroll через rAF

```
let ticking = false;
window.addEventListener('scroll', () => {
  if (ticking) return;
  ticking = true;
  requestAnimationFrame(() => {
    updateUI();
    ticking = false;
  });
}, { passive: true });
```

```
// content-visibility
```

```
.card {  
  content-visibility: auto;  
  contain-intrinsic-size: 0 200px;  
}
```

МОГУТ ДОСПРОСИТЬ

- 01 Что такое content-visibility?
- 02 Чем contain-intrinsic-size помогает?
- 03 Когда нужна виртуализация?

ПОЛНАЯ СТАТЬЯ

<https://web.dev/articles/content-visibility>

ФИНАЛЬНЫЙ ЧЕК - ЛИСТ

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ [] Описать Core Web Vitals: LCP / INP / CLS – что и какие пороги (Q1-Q4)
- ☐ [] Сравнить lab и field, объяснить RUM (Q5-Q7)
- ☐ [] Назвать критичные метрики для realtime/fintech UI (Q8)
- ☐ [] Описать, как найти long task / long animation frame (Q9, Q10)
- ☐ [] Объяснить browser rendering pipeline и layout thrashing (Q11, Q12, Q13)
- ☐ [] Расписать, что вызывает reflow / repaint / compositing (Q14, Q15, Q16)
- ☐ [] Сказать, какие CSS-свойства дешевле анимировать (Q17)
- ☐ [] Описать will-change и его подводные камни (Q18, Q19)
- ☐ [] Назвать 3 способа оптимизировать scroll performance (Q20)

EVENT LOOP / ASYNC МЕТОДИЧКА v1
МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru